

Full Marks:50

SET:A

Time: 1Hour 10Minutes

Q1. What will normally happen?

```
import math
import math
```

- A)The module is normally loaded and executed twice
- B)The module is normally loaded once and reused
- C)SyntaxError
- D)ImportError

Q2. What happens here?

```
from math import sqrt
```

```
print(sqrt(36))
```

- A)36
- B)6.0
- C)6
- D)NameError

Q3. What happens here?

```
class MyError(Exception):
    pass
```

```
def test():
    raise MyError("Something went wrong")
```

```
try:
    test()
except MyError as e:
    print(type(e).__name__)
```

- A)Exception
- B)TypeError
- C)Something went wrong
- D)MyError

Q4. What is the output?

```
try:
    x = int("abc")
except ValueError:
    print("A")
```

```
else:
    print("B")
finally:
    print("C")
```

- A)B
- B)A
- C)A C
- D)B C

Q5. Why is

```
try:
    print(10 / 0)
except Exception:
    print("A")
except ZeroDivisionError:
```

```
    print("B")
codes's second handler never reached?
```

- A)ZeroDivisionError occurs before Exception
- B)except blocks execute randomly
- C)Exception is a superclass of ZeroDivisionError
- D)ZeroDivisionError is not an exception

Q6. What will happen?

```
def fun(a, b=10):
    return a + b
```

```
print(fun(b=5, a=20))
```

- A)TypeError
- B)25
- C)15
- D)SyntaxError

Q7. if test.py has
print("A")

```
if __name__ == "__main__":
    print("B")
```

If test.py is executed directly, what will be printed?

- A)A followed by B
- B)B



C)A

D)Nothing

Q8. What will be printed?

```
def fun(x=[]):  
    x.append(10)  
    return x
```

a = fun()

b = fun()

print(a)

print(b)

A)Error

B)[10, 10]

[10]

C)[10]

[10]

D)[10]

[10, 10]

Q9. What is the output?

```
def fun(x):  
    return x * 2
```

a = fun

print(a(5))

A)Error

B)func(5)

C)10

D)5

Q10. What is the output?

```
def outer():  
    x = 10
```

```
    def inner():  
        nonlocal x  
        x = x + 5  
        return x
```

```
    inner()  
    return x
```

print(outer())

A)15

B)UnboundLocalError

C)10

D)5

Q11. What will be the output?

x = 10

```
def fun():  
    print(x)  
    x = 20
```

fun()

A)UnboundLocalError

B)NameError

C)10

D)20

Q12. What is the purpose of this statement?

```
if __name__ == "__main__":
```

A)It prevents syntax errors

B)It identifies code intended to run when the file is executed directly

C)It prevents all imports

D)It creates the main function

Q13. What is the output?

```
try:
```

```
    raise ValueError
```

```
except ValueError:
```

```
    print("A")
```

```
    raise
```

A)A, followed by propagation of the same exception

B)No Output

C)A, and execution ends normally

D)B

Q14. What will be printed?

```
def fun():
```

```
    try:
```

```
        return 10
```

```
    finally:
```

```
        print("Finally")
```

print(fun())

A)10 only

B)Finally followed by 10

C)10 followed by Finally

D)Finally only



Q15. What will be the output?

```
import math
```

```
print(math.sqrt(25))
```

- A) sqrt(25)
- B) 5.0
- C) 5
- D) 25

Q16. What will be printed?

```
x = 10
```

```
f = lambda: x
```

```
x = 20
```

```
print(f())
```

- A) None
- B) 10
- C) NameError
- D) 20

Q17. What will be printed?

```
x = 10
```

```
def outer():  
    x = 20
```

```
    def inner():  
        print(x)
```

```
    x = 30  
    inner()
```

```
outer()
```

- A) 20
- B) NameError
- C) 10
- D) 30

Q18. What will be printed?

```
def fun():  
    print("A")  
    return  
    print("B")
```

```
fun()
```

- A) B
- B) Nothing
- C) A B
- D) A

Q19. What will be the output?

```
x = 10
```

```
def outer():  
    x = 20
```

```
    def inner():  
        global x  
        x = 30
```

```
    inner()  
    print(x)
```

```
outer()  
print(x)
```

- A) 30
- 30
- B) 20
- 10
- C) 30
- 20
- D) 20
- 30

Q20. What is the output?

```
numbers = [1, 2, 3, 4, 5]
```

```
result = list(  
    map(  
        lambda x: x * 2,  
        filter(lambda x: x % 2 == 0,  
            numbers)  
    )  
)
```

```
print(result)
```

- A) [1, 3, 5]
- B) [2, 4]
- C) [4, 8]
- D) [2, 4, 6, 8, 10]

Q21. What is the output?

```
try:
```



```

print(10 / 0)
except Exception:
    print("A")
except ZeroDivisionError:
    print("B")
A)SyntaxError
B)B
C)A B
D)A

```

Q22. What will be printed?

```

def make_counter():
    count = 0

    def counter():
        nonlocal count
        count += 1
        return count

    return counter

```

```

a = make_counter()
b = make_counter()

```

```

print(a())
print(a())
print(b())
print(a())

```

- A)1
- 2
- 3
- 4
- B)0
- 1
- 0
- 2
- C)1
- 2
- 1
- 3
- D)1
- 1
- 1
- 1

Q23. What is the output?

```

def outer():
    x = 10

```

```

def inner():
    return x + 5

return inner

```

```

f = outer()
print(f())

```

- A)15
- B)None
- C)NameError
- D)10

Q24. Suppose the current directory contains a file named random.py. What potential problem can occur with:

```
import random
```

- A)random cannot be imported
- B)The local file can shadow the intended standard-library module
- C)Python automatically renames the local file
- D)Python will always reject it

Q25. What happens when this function is called?

```

def fun(a=10, b):
    return a + b

```

- A)SyntaxError
- B)It executes correctly
- C)NameError
- D)TypeError

Q26. What happens here?

```

def fun(a):
    return a

```

```
print(fun())
```

- A)NameError
- B)TypeError
- C)None
- D)0

Q27. What will happen?

```

def fun(n):
    if n == 0:
        return 1

```



```
return n * fun(n)
```

```
print(fun(3))
```

- A)TypeError
- B)3
- C)RecursionError
- D)6

Q28. What will happen?

```
def outer():
```

```
    x = 10
```

```
    def inner():
```

```
        nonlocal y
```

```
        y = 20
```

```
    inner()
```

```
outer()
```

- A)NameError
- B)SyntaxError
- C)20
- D)10

Q29. What does the following function return?

```
def fun():
```

```
    print("Hello")
```

```
x = fun()
```

```
print(x)
```

- A>Error
- B>Hello None
- C)None
- D>Hello

Q30. What happens?

```
def fun(a, b):
```

```
    return a + b
```

```
print(fun(10, a=20))
```

- A)SyntaxError
- B)20
- C)30
- D)TypeError

Q31. What is the output?

```
def fun(x):
```

```
    x += 10
```

```
    return x
```

```
x = 5
```

```
y = fun(x)
```

```
print(x, y)
```

- A)15 5
- B)5 5
- C)5 15
- D)15 15

Q32. What is the major difference between the following?

```
import math
```

```
and
```

```
from math import sqrt
```

- A)First provides the module namespace; second imports sqrt directly
- B)Both import only sqrt
- C)First imports only sqrt
- D)There is no difference

Q33. What will be printed?

```
class InvalidAgeError(Exception):
```

```
    pass
```

```
def check_age(age):
```

```
    if age < 18:
```

```
        raise InvalidAgeError("Age is too low")
    return "Allowed"
```

```
try:
```

```
    print(check_age(15))
```

```
except InvalidAgeError as e:
```

```
    print(type(e).__name__)
```

```
    print(e)
```

- A)Exception
- Age is too low
- B)InvalidAgeError
- 15



C)Nothing, because custom exceptions cannot be caught

D)InvalidAgeError

Age is too low

Q34. What is the output?

```
def fun():  
    try:  
        return 10  
    finally:  
        return 20
```

print(fun())

A)RuntimeError

B)20

C)None

D)10

Q35. What will be printed?

```
try:  
    print(10 / 0)  
except ZeroDivisionError:  
    print("A")  
except Exception:  
    print("B")
```

A)B

B)A B

C)Nothing

D)A

Q36. What will be printed?

from functools import reduce

numbers = [1, 2, 3, 4]

```
result = reduce(  
    lambda a, b: a * b,  
    numbers  
)
```

print(result)

A)10

B)24

C)9

D)0

Q37. What will be printed?

```
def fun(a, b):
```

return a + b

print(fun(10, b=20))

A)SyntaxError

B)TypeError

C)30

D)20

Q38. What will be the output?

```
def fun(x):  
    try:  
        if x < 0:  
            raise ValueError("negative")  
        return x  
    except ValueError:  
        return 100  
    finally:  
        print("cleanup")
```

print(fun(-5))

A)cleanup

100

B)100

cleanup

C)negative

cleanup

D)cleanup

-5

Q39. What happens here?

```
try:  
    raise ValueError("Wrong value")  
except ValueError as e:  
    print(e)
```

A)e

B)ValueError

C)Wrong Value

D)Nothing

Q40. What happens when this code executes?

```
def fun():  
    print(variable)  
    variable = 50
```

variable = 10

fun()

A)UnboundLocalError



B)50

C)NameError

D)10

Q41. What is the output?

```
def fun(x):  
    if x == 0:  
        return  
    return x + fun(x - 1)
```

```
print(fun(3))
```

A)6

B)3

C)None

D)5

Q42. What will be printed?

```
try:  
    x = int("10")  
except ValueError:  
    print("A")  
else:  
    print("B")  
finally:  
    print("C")
```

A)A C

B)B

C)A

D)B C

Q43. Suppose test.py contains:

```
print("A")
```

```
if __name__ == "__main__":  
    print("B")
```

If another Python program executes:

```
import test
```

what will normally be printed?

A)Nothing

B)A and B

C)A

D)B

Q44. Suppose module_a.py contains:

```
print("Module loaded")
```

```
value = 10
```

And another file contains:

```
import module_a  
import module_a
```

```
print(module_a.value)
```

Assuming both imports occur in the same Python process, what is normally printed?

A)Module loaded

Module loaded

10

B)ImportError

C)10

10

D)Module loaded

10

Q45. What happens when this code runs?

```
def first():  
    second()
```

```
def second():  
    third()
```

```
def third():  
    raise RuntimeError("Failure")
```

```
try:  
    first()  
except RuntimeError:  
    print("Handled")
```

A)Handled

B)NameError

C)RuntimeError cannot propagate through functions

D)TypeError

Q46. Which object keeps track of modules that have already been imported during the current Python process?

A)sys.path

B).__module__



C)sys.imports
D)sys.modules

Q47. What is the output?

```
def fun():  
    try:  
        print("A")  
        raise ValueError  
    except ValueError:  
        print("B")  
        return 10  
    else:  
        print("C")  
        return 20  
    finally:  
        print("D")
```

print(fun())

A)A
C
D
20
B)A
B
10
D
C)A
B
D
10
D)A
B
C
D
10

Q48. What is the output?

f = lambda x: x + 5

print(f(10))

A)10
B)15
C)5
D)Error

Q49. Suppose sample.py contains:

x = 10

```
def display():  
    print("Hello")
```

What will this print?

import sample

```
print(sample.x)  
sample.display()
```

A)Only Hello
B)10
Hello
C)x
display
D)Error

Q50. What is the output?

```
def fun(a=10, b=20):  
    return a - b
```

print(fun(30))

A)10
B)20
C)-10
D)30

